

US-001 — Create GitHub Repository Structure

Status

Completed

Objective

The objective of this User Story was to create the initial GitHub repository structure for the Expense Tracker application and prepare the repository for future frontend, backend, documentation, and feature development.

The following structure was targeted:

```
expense-tracker/  
|  
├─ frontend/  
├─ backend/  
├─ docs/  
├─ README.md  
└─ .gitignore
```

The Business Requirements Document (BRD) and Technical Design Document (TDD) were also added to the repository.

1. Initialize Git Repository

The project was already available locally, so Git was initialized from the project directory.

```
git init
```

Git successfully initialized a new local repository:

```
Initialized empty Git repository in C:/content-creation/fin-track-basic/  
expense-tracker/.git/
```

What this does

`git init` creates the `.git` directory and converts the current project directory into a Git repository.

At this point, the local repository existed, but it was not yet connected to GitHub.

2. Initial Branch Checkout Issue

The first attempt was to switch to an expected feature branch:

```
git checkout feature/expense-tracker-v0.1
```

Git returned:

```
error: pathspec 'feature/expense-tracker-v0.1' did not match any file(s)
known to git
```

I also tried:

```
git checkout main
```

and:

```
git checkout master
```

Both failed because the local repository had no branches yet.

Why this happened

`git init` created a completely new local repository.

There were no local branches because no commits had been created yet, and the local repository was not connected to the GitHub repository.

Therefore Git could not find:

```
main
master
feature/expense-tracker-v0.1
```

3. Initial Git Pull Issue

I then tried:

```
git pull origin master
```

This resulted in:

```
fatal: 'origin' does not appear to be a git repository
```

Why this happened

The remote repository had not been configured yet.

Git did not know what `origin` meant.

To verify this, I ran:

```
git remote -v
```

There was no output.

This confirmed that no remote repository was configured.

4. Correct the Git Remote Configuration

The GitHub repository was then added as the `origin` remote:

```
git remote add origin https://github.com/jaydeeprajputprojects-wq/expense-tracker.git
```

I verified the configuration:

```
git remote -v
```

Result:

```
origin https://github.com/jaydeeprajputprojects-wq/expense-tracker.git
(fetch)
origin https://github.com/jaydeeprajputprojects-wq/expense-tracker.git
(push)
```

Why `origin` is used

`origin` is the conventional name given to the main remote repository.

It allows commands such as:

```
git fetch origin
git pull origin main
git push origin main
```

5. Fetch Remote Branches

After configuring the remote, I fetched the branches available on GitHub:

```
git fetch origin
```

Then checked all branches:

```
git branch -a
```

The result showed:

```
remotes/origin/main
```

This confirmed that the GitHub repository already had a `main` branch.

6. Create Local Feature Branch from Remote Main

Instead of working directly on `main`, I created the project feature branch from the existing remote `main`:

```
git checkout -b feature/expense-tracker-v0.1 origin/main
```

Git returned:

```
Switched to a new branch 'feature/expense-tracker-v0.1'
branch 'feature/expense-tracker-v0.1' set up to track 'origin/main'.
```

I verified the active branch:

```
git branch
```

Result:

```
* feature/expense-tracker-v0.1
```

Why this was done

The feature branch was created so that the initial project setup could be developed independently instead of directly modifying `main`.

The branch was initially based on:

```
origin/main
```

7. Initial Commit of Existing Documentation

The existing project documentation was staged:

```
git add .
```

Git displayed line-ending warnings:

```
LF will be replaced by CRLF the next time Git touches it
```

What this means

The files used Unix-style `LF` line endings while Windows commonly uses `CRLF`.

Git was warning that the line endings may be converted when Git modifies the files in the Windows working environment.

This was a warning, not a failure, and the files were successfully staged.

The initial documentation commit was created:

```
git commit -m "initial repo setup"
```

Commit:

```
6d19db5
```

The commit included the project documentation.

8. First Push / Upstream Branch Issue

I then attempted:

```
git push
```

Git returned:

```
fatal: The upstream branch of your current branch does not match  
the name of your current branch.
```

Why this happened

The local branch was:

```
feature/expense-tracker-v0.1
```

but it had initially been configured to track:

```
origin/main
```

Therefore Git's default push behavior could not automatically determine the intended destination.

Git suggested:

```
git push origin HEAD
```

I used the recommended approach later.

9. Verify Remote and Git User Configuration

I verified the remote again:

```
git remote -v
```

and checked the current Git branch:

```
git branch --show-current
```

Result:

```
feature/expense-tracker-v0.1
```

I also checked the Git identity:

```
git config --global user.name
```

Result:

```
Jaydeep Rajput
```

I initially entered an email incorrectly:

```
git config --global jaydeeprajputprojects@gmail.com
```

Git returned:

```
error: invalid key: jaydeeprajputprojects@gmail.com
```

Why this happened

Git configuration commands require a configuration key.

The correct format is:

```
git config --global user.email "email@example.com"
```

So I corrected it using:

```
git config --global user.email "jaydeeprajputprojects@gmail.com"
```

Then verified:

```
git config --global user.email
```

Result:

```
jaydeeprajputprojects@gmail.com
```

10. GitHub 403 Permission Error

When I tried to push the branch:

```
git push -u origin HEAD
```

GitHub returned:

```
remote: Permission to jaydeeprajputprojects-wq/expense-tracker.git denied to
jaydeeprajput.
fatal: unable to access ...
The requested URL returned error: 403
```

Why this happened

The Git remote URL was correct, but GitHub authentication was using credentials associated with a different GitHub account:

```
jaydeeprajput
```

while the repository belonged to:

```
jaydeeprajputprojects-wq
```

Changing:

```
git config user.name
git config user.email
```

does **not** change the GitHub account used for authentication.

This was an important distinction discovered during troubleshooting.

Important learning

Git has two separate concepts:

Commit identity

Configured using:

```
git config --global user.name  
git config --global user.email
```

This controls the author information attached to commits.

GitHub authentication

This controls whether GitHub allows the local machine to push to a repository.

Changing `user.name` or `user.email` alone does not solve an authentication/permission problem.

11. Resolve GitHub Authentication

When pushing again:

```
git push -u origin feature/expense-tracker-v0.1
```

Git displayed:

```
info: please complete authentication in your browser...
```

I completed the GitHub authentication through the browser.

After authentication, I repeated:

```
git push -u origin feature/expense-tracker-v0.1
```

This time the push succeeded.

Git reported:

```
[new branch] feature/expense-tracker-v0.1 -> feature/expense-tracker-v0.1
```

The local feature branch was now available on GitHub.

12. Create Required Project Directories

After resolving the GitHub connectivity issue, the required project structure was created.

```
mkdir frontend  
mkdir backend  
mkdir docs
```

The resulting structure was:

```
expense-tracker/  
├──  
├── frontend/  
├── backend/  
└── docs/
```

13. Add Frontend and Backend README Files

README files were added to the frontend and backend directories:

```
New-Item frontend/README.md -ItemType File
```

```
New-Item backend/README.md -ItemType File
```

This ensures Git can track the directories and provides a placeholder for future module-specific documentation.

14. Add Documentation

The BRD and TDD documentation were placed under:

```
docs/
```

The structure became:

```
docs/  
├── Personal Finance & Money Flow Tracker – Business Requirements Document.md  
└── Personal Finance & Money Flow Tracker – Technical Design Document.md
```

This keeps project requirements and technical architecture together with the source code.

15. Validate Project Structure

The project structure was verified using:

```
tree /F
```

The resulting structure included:

```
C:.\
|  README.md
|
|  └─ backend
|      README.md
|
|  └─ docs
|      Personal Finance & Money Flow Tracker – Business Requirements
|      Document.md
|      Personal Finance & Money Flow Tracker – Technical Design Document.md
|
|  └─ Documentation
|      Personal Finance & Money Flow Tracker – Business Requirements
|      Document.md
|      Personal Finance & Money Flow Tracker – Business Requirements.pdf
|      Personal Finance & Money Flow Tracker – Technical Design Document.md
|      Personal Finance & Money Flow Tracker.pdf
|
|  └─ frontend
|      README.md
```

Observation

The repository currently contains both:

```
docs/
```

and:

```
Documentation/
```

The `docs/` directory contains the Markdown BRD/TDD copies, while the existing `Documentation/` directory contains the original documentation files/PDFs.

This should be reviewed in a future cleanup/refactoring story if the project requires a single documentation location.

16. Stage New Changes

The newly created structure and documentation were staged:

```
git add .
```

Git showed the files that were going to be committed.

The staged files included:

```
backend/README.md
frontend/README.md
docs/Personal Finance & Money Flow Tracker – Business Requirements
Document.md
docs/Personal Finance & Money Flow Tracker – Technical Design Document.md
Documentation/...
```

17. Commit Repository Structure

A proper repository initialization commit was created:

```
git commit -m "chore: initialize project repository structure"
```

Commit:

```
e1fb039
```

The commit contained:

```
6 files changed
4899 insertions
```

18. Push Final Changes

The changes were pushed to the feature branch:

```
git push
```

Git successfully pushed:

```
6d19db5..e1fb039
feature/expense-tracker-v0.1 -> feature/expense-tracker-v0.1
```

At this point the repository structure and project documentation were available on GitHub.

Key Problems Encountered and Solutions

Problem	Root Cause	Solution
Branch not found	Local repository had no branches	Added remote and fetched branches
<code>main</code> / <code>master</code> not found	Repository was freshly initialized	Fetched <code>origin/main</code>
<code>origin</code> not found	Remote was not configured	<code>git remote add origin ...</code>
<code>git remove -v</code> failed	Incorrect Git command	Used <code>git remote -v</code>
Push upstream mismatch	Local branch tracked <code>origin/main</code>	Used <code>git push -u origin HEAD</code> / explicit branch
Invalid Git config key	Email was supplied without <code>user.email</code>	Used <code>git config --global user.email ...</code>
GitHub 403	Wrong GitHub authentication/account	Completed browser authentication
LF → CRLF warning	Windows/Linux line-ending difference	Accepted warning; commit succeeded

Git Commands Learned

The following Git commands were used during this User Story:

```
git init
git checkout
git remote -v
git remote add origin <repository-url>
git fetch origin
git branch -a
```

```
git checkout -b <branch> origin/main
git branch
git branch --show-current
git status
git add .
git commit
git push
git push -u origin HEAD
git config --global user.name
git config --global user.email
git pull
```

Final Repository State

The initial repository foundation was successfully created.

```
expense-tracker/
├── frontend/
│   └── README.md
├── backend/
│   └── README.md
├── docs/
│   ├── Business Requirements Document.md
│   └── Technical Design Document.md
├── Documentation/
│   ├── BRD.md
│   ├── BRD.pdf
│   ├── TDD.md
│   └── TDD.pdf
└── README.md
```

The feature branch:

```
feature/expense-tracker-v0.1
```

was successfully pushed to GitHub.

The repository is now ready for the next development User Story.

Lessons Learned

1. `git init` does not connect a project to GitHub

Initializing Git only creates a local repository.

You still need:

```
git remote add origin <url>
```

2. Local Git identity and GitHub authentication are different

This:

```
git config --global user.email
```

controls commit metadata.

It does not determine which GitHub account is authenticated for pushing.

3. Always check the remote

When something goes wrong with push/pull:

```
git remote -v
```

is one of the first commands to run.

4. Check branches before checkout

Use:

```
git branch -a
```

to see both local and remote branches.

5. `git fetch` is useful before creating a branch

Fetching the remote first allowed the local repository to see:

```
origin/main
```

and create the feature branch correctly.

6. Browser authentication can resolve GitHub credential problems

When Git reported:

```
please complete authentication in your browser
```

completing the authentication allowed the push to succeed.

Definition of Done

- [x] GitHub repository exists
- [x] Local Git repository initialized
- [x] GitHub remote configured
- [x] Remote branches fetched
- [x] Feature branch created
- [x] Frontend directory created
- [x] Backend directory created
- [x] Docs directory created
- [x] Frontend README created
- [x] Backend README created
- [x] BRD added
- [x] TDD added
- [x] Repository structure verified
- [x] Initial repository structure committed
- [x] GitHub authentication resolved
- [x] Feature branch pushed to GitHub

Final Status:  **US-001 COMPLETED**